NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPPALLI

Department of Computer Science and Engineering

CryptoChunk

Distributed File Chunking with Erasure Coding and Cryptographic Checksum Verification Across Untrusted Storage Nodes

Mini Project Report

Subject: Advanced Cryptography

Name: Krishna Mahajan
Roll No.: 106123070
Department: CSE
Institute: NIT Trichy

Submitted as an academic mini-project implementation.

Abstract

This project implements CryptoChunk, an educational prototype for storing files as fixed-size logical chunks across untrusted storage locations. Each logical chunk is encoded into six shards using a systematic 4+2 erasure code over the finite field $GF(2^8)$. The four data shards preserve the original information, while two parity shards allow reconstruction if any two shard files are missing or fail integrity checks. SHA-256 digests are recorded for each shard, each original logical chunk, and the complete file. Since unkeyed hashes do not authenticate metadata, the manifest is additionally authenticated using HMAC-SHA-256. During restoration, the program verifies the manifest, classifies missing or corrupted shards, reconstructs from at least four valid shards, checks the reconstructed content, and repairs damaged shard files. Tests cover all 15 choices of four out of six shards, recovery after simulated faults, manifest tampering, and failure when more than two shards are lost. The implementation uses six local directories to model storage nodes; it is not presented as a production networked storage service.

Project Objectives

1. Implement fixed-size file chunking and a repeatable shard placement model.
2. Add erasure coding to tolerate unavailable or corrupted storage objects.
3. Verify stored data using cryptographic checksums.
4. Authenticate metadata so an attacker cannot silently rewrite file sizes, chunk ordering, or expected digests without the key.
5. Evaluate recovery behaviour, integrity checks, and storage overhead with repeatable tests and a benchmark script.

Contents

1 Introduction **1**

2 Requirements and Scope **2**

2.1 Functional requirements 2

2.2 Explicit scope boundary 2

3 System Design **3**

3.1 Architecture 3

3.2 Repository structure 3

3.3 Chunking and shard placement 4

3.4 Erasure code 4

3.5 Integrity and authenticity 4

3.6 Threat model 4

4 Implementation **6**

4.1 Storage operation 6

4.2 Restore operation 6

4.3 Command examples 6

5 Evaluation **7**

5.1 Test strategy and results 7

5.2 Benchmark methodology 7

6 Security Analysis and Limitations **9**

6.1 Security properties achieved 9

6.2 Limitations and next steps 9

7 Conclusion **10**

A Reproducibility Notes **12**

List of Tables

5.1 Automated test results from the project build. 7
5.2 Measured local benchmark results. 8

Chapter 1

Introduction

Distributed storage improves availability by keeping information across multiple storage locations, but storage nodes may fail, lose objects, or return modified bytes. Replication is one way to handle failures; erasure coding can achieve redundancy with less storage overhead than storing multiple complete copies. A separate integrity mechanism is also necessary: erasure coding helps recover data but does not, by itself, establish whether the bytes returned by a node are the expected bytes.

CryptoChunk combines these ideas in a compact, inspectable implementation. The prototype splits an input into logical chunks, encodes each chunk into four data shards and two parity shards, places one shard per node directory, and records cryptographic digests in an authenticated manifest. The SHA-256 construction follows the Secure Hash Standard described by NIST [1]. HMAC is used to authenticate the manifest; the Python HMAC API recommends constant-time comparison through `compare_digest` when verifying externally supplied tags [2].

Chapter 2

Requirements and Scope

2.1 Functional requirements

- Store a file in logical chunks with a configurable chunk size.
- Generate six equal-length shards per logical chunk using a systematic 4+2 code.
- Store shards in six separate local node directories.
- Record SHA-256 digests, original lengths, coding parameters, and file metadata.
- Authenticate the manifest using HMAC-SHA-256.
- Restore a file using any four valid shards per chunk and detect unrepairable loss.
- Provide fault injection for missing and modified shards, plus automated tests and a benchmark.

2.2 Explicit scope boundary

The current deliverable simulates independent storage nodes as local directories on one host. It does not implement network transport, a node discovery service, consensus, remote authentication, encryption at rest, or production key custody. The manifest key is stored beside the manifest to make the demonstration easy to run; a real system must keep the authentication key outside the storage attacker's control. These limits are important to the security interpretation of the results.

Chapter 3

System Design

3.1 Architecture

The implementation is separated into three layers:

1. **Erasure layer:** `erasure.py` implements arithmetic in $GF(2^8)$, the generator matrix, encoding, matrix inversion, and recovery.
2. **Storage and integrity layer:** `engine.py` chunks files, writes shards, builds and authenticates metadata, verifies shard hashes, reconstructs data, and repairs bad shard files.
3. **Command-line layer:** `cli.py` exposes store, restore, and fault-injection commands.

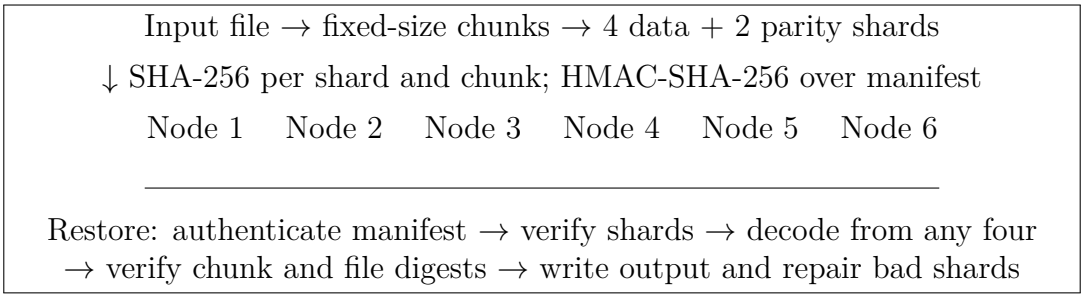


Figure 3.1: CryptoChunk processing and recovery flow.

3.2 Repository structure

```
cryptochunk-project/
  src/cryptochunk/        erasure.py, engine.py, cli.py
  tests/                  automated unit and integration tests
  examples/               sample input and demo shell script
  scripts/benchmark.py    reproducible benchmark
  artifacts/benchmark.csv measured output from this run
  docs/project_report.tex report source
```

```
README.md              setup, commands, security scope
pyproject.toml         packaging and CLI entry point
```

3.3 Chunking and shard placement

For a logical chunk of L bytes, the encoder pads the chunk to a multiple of four equal data-shard lengths. Let $s = \max(1, \lceil L/4 \rceil)$; the four data shards each contain s bytes. The final decoded byte stream is truncated to the authenticated original length L. The six shard indexes $0, \ldots, 5$ map to six node directories. Each manifest record includes the relative path, size, shard index, node identifier, and SHA-256 digest.

3.4 Erasure code

The prototype uses a systematic generator matrix over $GF(2^8)$:

$$G = \begin{bmatrix} I_4 \\ P \end{bmatrix}, \qquad P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{bmatrix}.$$

The field is represented using the primitive polynomial $x^8 + x^4 + x^3 + x^2 + 1$ (hexadecimal representation `0x11D`). Field addition is bitwise XOR; multiplication and inversion use exponent/log tables. The first four rows are the identity matrix, so the code is systematic: the original data shards remain directly represented in the output. The two parity rows provide redundancy. Any four of the six rows form an invertible matrix, enabling reconstruction from any four valid shard indexes. This matches the standard erasure-code objective of reconstructing data when shard locations are known to be unavailable; the project uses the `reedsolo` documentation as a conceptual reference for Reed–Solomon coding and errors/erasures terminology, but does not depend on that package at runtime [3].

3.5 Integrity and authenticity

SHA-256 is calculated over every stored shard, each original chunk, and the complete input file. During restore, a shard is accepted only if its size and digest match the manifest. The manifest itself is serialized as canonical JSON (sorted keys and compact separators) and authenticated with HMAC-SHA-256 using a random 256-bit key. Verification uses `hmac.compare_digest`. This separation is deliberate: a digest can detect altered data only when the expected digest is trusted; a keyed MAC protects the manifest against an attacker who can write storage objects but does not possess the key.

3.6 Threat model

The prototype considers nodes that may be unavailable, omit a shard, truncate it, or return modified bytes. The client trusts the manifest authentication key and the local

execution environment. If at least four of six shards per chunk remain valid, recovery is expected to succeed. If fewer than four valid shards remain, the implementation fails closed and reports the affected chunk. An attacker who can replace both the manifest and the HMAC key can defeat this demo's metadata protection; this is documented as a limitation rather than treated as a secure deployment design.

Chapter 4

Implementation

4.1 Storage operation

The `store` command reads the source, divides it into logical chunks, generates six shards per chunk, writes each shard to its node directory, and creates a manifest containing the file identifier, original size, full-file digest, chunk configuration, per-chunk digest, and per-shard metadata. The manifest is written with an HMAC tag. The CLI emits a JSON summary including chunk count, number of shards, stored bytes, and elapsed time.

4.2 Restore operation

Restore first loads the key and authenticates the manifest. For every shard record, it checks file presence, expected length, and SHA-256. Missing or corrupt shards are excluded from the available set. If fewer than four valid shards remain for a chunk, restoration stops with an error. Otherwise, the selected generator rows are inverted over $GF(256)$ and the four data shards are recovered. The original chunk length is applied, and the chunk digest is checked. After all chunks are concatenated, both the full-file length and digest are checked before the output is written. When a shard was missing or corrupt, its bytes are regenerated from the recovered chunk and written back to the expected location.

4.3 Command examples

```
python -m pip install -e .
cryptochunk store input.bin --storage ./storage --chunk-size
   262144
cryptochunk fault --storage ./storage --chunk 0 --shard 1 --
   action corrupt
cryptochunk fault --storage ./storage --chunk 0 --shard 4 --
   action remove
cryptochunk restore --storage ./storage --output ./restored.bin
pytest -q
python scripts/benchmark.py
```

Chapter 5

Evaluation

5.1 Test strategy and results

The automated test suite includes exhaustive selection of each possible four-shard subset, an empty-file coding edge case, an integration test that corrupts one shard and removes another, manifest-tampering rejection, and expected failure when three shards are unavailable in a chunk.

Table 5.1: Automated test results from the project build.

Test group	Result
All 15 combinations of four valid shards	PASS
Empty input handling	PASS
Corruption + removal recovery and file equality	PASS
Modified manifest authentication	PASS
Three-shard loss rejection	PASS
Total	5 tests passed

5.2 Benchmark methodology

The included benchmark generates deterministic byte sequences of 1 KiB, 64 KiB, 1 MiB, and 4 MiB. Each input is stored with a 256 KiB logical chunk size; one shard is corrupted and another is removed before restoration. Timing uses Python's `perf_counter`. The CSV records Python version, platform, stored bytes, overhead, store/restore elapsed time, SHA-256, and whether the restored bytes equal the source. Timings are specific to the execution environment and are not claims about production network throughput.

All four benchmark fault-tolerance checks passed. The storage ratio is six shard bytes per four bytes of padded data, giving a theoretical 1.5x storage factor (50% overhead over original data for these tested sizes). For very small or non-divisible inputs, padding can increase the effective overhead. Metadata overhead is not included in the table. The implementation is pure Python, so its throughput is suitable for a teaching prototype rather than a high-performance storage engine.

Table 5.2: Measured local benchmark results.

Input size	Chunks	Stored bytes	Overhead	Store (s)	Restore (s)
1 KiB	1	1,536	50.0%	0.002595	0.000761
64 KiB	1	98,304	50.0%	0.020573	0.027504
1 MiB	4	1,572,864	50.0%	0.226400	0.194535
4 MiB	16	6,291,456	50.0%	0.903344	0.503957

Chapter 6

Security Analysis and Limitations

6.1 Security properties achieved

- **Integrity detection:** per-shard SHA-256 identifies altered shard bytes relative to trusted metadata.
- **Metadata authenticity:** HMAC-SHA-256 prevents undetected manifest modification when the key remains secret.
- **Availability under loss:** the 4+2 code recovers a chunk from any four valid shards.
- **Post-recovery verification:** chunk and full-file digests validate the recovered bytes before reporting success.

6.2 Limitations and next steps

1. **Local-node simulation:** implement a network protocol and deploy nodes on separate processes or hosts to demonstrate actual distribution.
2. **Key management:** keep the HMAC key client-side or in a dedicated secret store; never store it alongside untrusted metadata in production.
3. **Confidentiality:** erasure coding does not encrypt content. Add an established authenticated-encryption scheme before shard placement when confidentiality is required.
4. **Streaming and scalability:** process chunks incrementally rather than reading the whole file into memory; use atomic writes and crash-safe manifest replacement.
5. **Operational controls:** add access control, node health monitoring, retry policy, quotas, and concurrent transfers.
6. **Cryptographic validation:** the hand-written finite-field code is educational; a production deployment should use a reviewed, tested erasure-coding implementation and a well-defined interoperability profile.

Chapter 7

Conclusion

CryptoChunk demonstrates the interaction between erasure coding and cryptographic integrity in a small, testable codebase. Fixed-size logical chunks are encoded as six shards, and any four valid shards are sufficient for reconstruction. SHA-256 checksums detect corrupted storage objects, while HMAC-SHA-256 authenticates the metadata needed to interpret those checksums. The tests confirm recovery from one corrupted and one missing shard, rejection of tampered metadata, and explicit failure beyond the two-shard fault tolerance. The benchmark confirms the expected 50% raw storage overhead for the tested 4+2 configurations. The next major step toward a true distributed system is moving the six local node directories behind authenticated remote storage services and separating key management from the storage layer.